

PROJET DE POO IATIC3

Rapport Final Projet The Island
Version informatique du jeu de société

Groupe 5

- **Najlae BETTACHE**
- **Nathanaël FOUODJI DJOUDA**
- **Mamadou Sadio DIALLO**
- **Boubacar BOULKASSOUM**
- **Uriel Ange Javit Uriel**

Encadrant : Dhekra ABOUDA

Table des matières

Table des matières	2
Introduction	4
I. Présentation du Projet.....	4
II. Architecture générale du projet	4
1. Organisation en couches.....	4
2. Dépendances entre packages	5
3. Point d'entrée.....	5
III.Modèle de domaine détaillé	6
1. Représentation du plateau	6
2. Les tuiles de terrain.....	6
3. Les explorateurs et les bateaux	7
4. L'état global de la partie.....	7
5. Le dé de créature	7
IV.Services métier	7
1. Déplacements (MovementService).....	7
2. Retrait des tuiles (TileRemovalService).....	8
3. Gestion des créatures (CreatureManager).....	8
4. Tuiles en main (InteractionController).....	8
V. Interface graphique (Swing)	9
1. Écran d'accueil (WelcomeFrame).....	9
2. Plateau de jeu (InteractiveBoardPanel).....	9
3. Contrôleur d'interaction (InteractionController).....	9
4. Panneau d'information et aide.....	9
5. Bouton Placement automatique.....	10
VI.Illustrations de l'interface	10
VII. Tests unitaires.....	13
VIII. Manuel utilisateur	14
1. Prérequis et lancement	14
2. Démarrage du jeu	14
3. Déroulement d'une partie	14
a. Préparation – Placement des explorateurs.....	14
b. Préparation – Placement des bateaux	14
c. Tour de jeu (Phase 1 – Tuile de la main)	14
d. Tour de jeu (Phase 2 – Déplacements).....	14
e. Tour de jeu (Phase 3 – Retrait de tuile).....	14
f. Tour de jeu (Phase 4 – Dé de créature)	15
g. Fin de partie	15

IX. Planning et répartition des tâches	15
1. Découpage en phases de développement.....	15
2. Répartition des tâches par membre.....	15
3. Outils de développement.....	16
X. Problèmes rencontrés et solutions apportées.....	16
XI. Bilan	16
1. Fonctionnalités implémentées.....	16
2. Qualité du code.....	17
Conclusion.....	18

Introduction

Ce rapport présente l'implémentation réalisée dans le cadre du projet de programmation orientée objet de troisième année de cycle ingénieur informatique à l'ISTY. L'objectif était de concevoir et développer une version informatique jouable du jeu de société *The Island*, en Java, en respectant les contraintes techniques imposées par le cahier des charges.

I. Présentation du Projet

Le jeu de société *The Island* met en scène des explorateurs tentant de fuir une île qui s'enfonce progressivement dans la mer sous l'effet d'un volcan. Les joueurs doivent acheminer leurs explorateurs à bord de bateaux vers des îles refuges, tout en gérant les attaques des créatures marines (serpents de mer, requins et baleines) et les effets des tuiles de terrain retiré. La partie se joue à 4 joueurs humains sur une seule machine, conformément aux exigences de l'énoncé du sujet : « *le programme fonctionnera sur une seule machine (pas en réseau)* ». Ce document décrit l'architecture logicielle adoptée, le modèle objet retenu, les services métier implémentés, l'interface graphique développée en Swing, les tests unitaires réalisés, ainsi que le bilan organisationnel du projet.

II. Architecture générale du projet

1. Organisation en couches

Le projet adopte une architecture en couches distinctes, organisée autour du package racine `fr.iatic.theisland`. Chaque couche possède une responsabilité unique et ses dépendances sont orientées vers les couches inférieures, respectant ainsi le principe de séparation des responsabilités.

Tableau 1: Structure des packages et classes (package racine : `fr.iatic.theisland`)

Package	Rôle	Classes principales
<code>fr.iatic.theisland</code>	Point d'entrée	<code>App</code>
<code>fr.iatic.theisland.model</code>	Modèle main	<code>Board</code> , <code>HexCell</code> , <code>HexCoordinate</code> , <code>Explorer</code> , <code>ExplorerStatus</code> , <code>Boat</code> , <code>BoatStatus</code> , <code>Creature</code> , <code>CreatureType</code> , <code>Dice</code> , <code>GameState</code> , <code>PieceState</code> , <code>Player</code> , <code>PlayerColor</code> , <code>TerrainTile</code> , <code>TerrainType</code> , <code>TileEffect</code> , <code>BaseCellType</code>
<code>fr.iatic.theisland.setup</code>	Initialisation	<code>BoardFactory</code> , <code>GameSetupFactory</code> , <code>TerrainTileFactory</code> , <code>InitialPlacementService</code> , <code>AutoSetupService</code>
<code>fr.iatic.theisland.movement</code>	Déplacements	<code>MovementService</code> , <code>MovementContext</code> , <code>BoatControlService</code> , <code>ActionResult</code>

fr.iatic.theisland.engine	Moteur de jeu	CreatureManager, TileRemovalService
fr.iatic.theisland.ui	Interface Swing	MainFrame, WelcomeFrame, InteractivePanel, InteractiveBoardPanel, BoardPanel, LegendPanel, InteractionController, DemoPhase, SelectionKind

2. Dépendances entre packages

Les dépendances sont organisées de manière à éviter tout couplage circulaire :

- ❖ fr.iatic.theisland.ui dépend de model, movement et engine
- ❖ fr.iatic.theisland.movement dépend uniquement de model
- ❖ fr.iatic.theisland.engine dépend uniquement de model
- ❖ fr.iatic.theisland.setup dépend uniquement de model
- ❖ fr.iatic.theisland(App.java) orchestre ui et setup

Aucune couche inférieure ne dépend de ui, ce qui permet une réutilisation du modèle et des services dans d'éventuels futurs contextes.

3. Point d'entrée

Le point d'entrée est la classe App. Elle initialise le look-and-feel Nimbus (avec repli silencieux si indisponible), puis lance WelcomeFrame – l'écran d'accueil – dans le thread Swing via SwingUtilities.invokeLater.

```

1 package fr.iatic.theisland;
2
3 import fr.iatic.theisland.ui.WelcomeFrame;
4
5 /**
6  * Point d'entrée du jeu complet.
7  */
8 public final class App {
9
10     private App() {
11         // Classe utilitaire.
12     }
13
14     public static void main(String[] args) {
15         try {
16             UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel");
17         } catch (Exception ignored) {
18             // Si Nimbus n'est pas disponible, Swing utilise le thème par défaut.
19         }
20
21         SwingUtilities.invokeLater(() -> {
22             WelcomeFrame frame = new WelcomeFrame();
23             frame.setVisible(true);
24         });
25     }
26 }

```

Figure 1: Point d'entrée principal (App.java)

III. Modèle de domaine détaillé

1. Représentation du plateau

Le plateau est modélisé par la classe Board, qui encapsule une grille bidimensionnelle de cases hexagonales HexCell. Chaque case est identifiée par une instance immuable de HexCoordinate(ligne, colonne).

La topologie hexagonale utilise un décalage pair/impair en colonnes pour calculer les six voisins d'une case. La méthode getNeighbors() retourne une liste immuable des voisins valides.

Le plateau distingue trois catégories de cases via l'énumération BaseCellType:

- SEA: cases de mer ordinaires.
- RESCUE_ISLAND: îles-refuges situées dans les quatre coins du plateau.
- LAND: emplacements des tuiles de terrain formant l'île centrale.

2. Les tuiles de terrain

La classe TerrainTile encapsule le type de terrain (TerrainType : BEACH, FOREST, MOUNTAIN) et l'effet caché (TileEffect). Les 40 tuiles sont réparties selon la distribution officielle du jeu :

Tableau 2: Effets des tuiles de terrain (TileEffect)

Effet	Type	Contour	Description
SHARK_APPEARS	Vert	Immédiat	Place un requin ; tue les nageurs présents
WHALE_APPEARS	Vert	Immédiat	Place une baleine sur la case
BOAT_APPEARS	Vert	Immédiat	Place un bateau ; embarque les nageurs
WHIRLPOOL	Vert	Immédiat	Retire tout sur la case et ses voisines
VOLCANO	Vert	Immédiat	Fin de partie immédiate
DOLPHIN	Rouge	Début de tour	Déplace un nageur de 1 à 3 cases
WIND	Rouge	Début de tour	Déplace un bateau contrôlé de 1 à 3 cases
MOVE_SERPENT	Rouge	Début de tour	Téléporte un serpent vers n'importe quelle case de mer
MOVE_SHARK	Rouge	Début de tour	Téléporte un requin vers n'importe quelle case de mer
MOVE_WHALE	Rouge	Début de tour	Téléporte une baleine vers n'importe quelle case de mer
CANCEL_SHARK	Rouge	Défense	Retire un requin attaquant vos nageurs
CANCEL_WHALE	Rouge	Défense	Retire une baleine attaquant votre bateau

La fabrique TerrainTileFactory crée les 40 tuiles selon la répartition officielle (16 plages, 16 forêts, 8 montagnes) et les mélange aléatoirement via Collections.shuffle.

3. Les explorateurs et les bateaux

La classe `Explorer` modélise un pion appartenant à un joueur, porteur d'une valeur de trésor de 1 à 6. L'état est suivi via l'énumération `ExplorerStatus`:

- ❖ `UNPLACED`: pas encore posé sur le plateau.
- ❖ `ON_LAND`: sur une tuile de terrain.
- ❖ `SWIMMER`: dans l'eau (nageur).
- ❖ `IN_BOAT`: passager d'un bateau.
- ❖ `SAVED`: mis en sécurité sur une île-refuge.
- ❖ `REMOVED`: éliminé du jeu (attaque de créature ou tourbillon).

Chaque transition valide l'état précédent et lève une `IllegalStateException` si l'appel est illégal (principe de l'objet gardant son invariant).

La classe `Boat` applique la contrainte de capacité maximale de 3 passagers (`MAX_PASSENGERS = 3`). La logique de contrôle par majorité est isolée dans `BoatControlService`.

4. L'état global de la partie

La classe `PieceState` centralise les listes mutables (explorateurs, bateaux) et constitue la source de vérité unique transmise à tous les services. La classe `GameState` gère l'état de haut niveau : créatures sur le plateau, stocks (6 requins, 5 baleines, 4 bateaux bonus), tuiles en main par joueur, et fin de partie avec calcul des scores.

5. Le dé de créature

La classe `Dice` implémente un dé à 3 faces (résultat aléatoire uniforme) : `SERPENT`, `REQUIN` ou `BALEINE`. Il n'y a pas de face neutre dans cette implémentation ; tout lancer produit un type de créature.

```

23
24 public int getMaxMoves() {
25     return switch (type) {
26         case SERPENT -> 1;
27         case REQUIN -> 2;
28         case BALEINE -> 3;
29     };
30 }
31

```

Figure 2: Dé de créature (`Dice.java`):

IV. Services métier

1. Déplacements (`MovementService`)

Le service `MovementService` implémente l'intégralité des règles de déplacement. Il expose des méthodes distinctes pour chaque type de mouvement légal. Chaque méthode retourne un `ActionResult` indiquant le succès ou l'échec avec un message explicite destiné à l'interface.

Un `MovementContext` est créé en début de tour ; il maintient les 3 points de déplacement disponibles et les décrémentes à chaque action. Les mouvements autorisés sont :

- ❖ Explorateur sur terre : case terrain adjacente, bateau adjacent, ou mer adjacente (devient nageur).
- ❖ Nageur : 1 case de mer par tour ; montée dans un bateau uniquement si même case.

- ❖ Passager : débarquement sur abri (sauvé), transfert vers autre bateau adjacent, ou saut à la mer.
- ❖ Bateau : 1 case de mer adjacente si le joueur le contrôle.
- ❖ Une fois qu'un explorateur a quitté l'île, il ne peut plus y retourner.

2. Retrait des tuiles (TileRemovalService)

- Le TileRemovalService applique l'ordre de retrait des tuiles : les tuiles plage doivent être retirées en premier, puis les tuiles forêt, puis les tuiles montagne. Il vérifie aussi qu'une tuile est adjacente à au moins une case de mer avant d'autoriser son retrait.
- La validation utilise le plateau pour compter les tuiles restantes de chaque type. Ainsi, une tuile forêt ne peut pas être retirée tant qu'il reste encore une tuile plage, et une tuile montagne ne peut pas être retirée tant qu'il reste encore une tuile plage ou forêt.
- Lorsqu'une tuile est retirée, son effet caché est lu à partir de l'énumération TileEffect puis traité directement par TileRemovalService. Selon l'effet obtenu, le service peut ajouter une créature, ajouter un bateau, appliquer un tourbillon, déclencher le volcan ou conserver une tuile spéciale dans la main du joueur.
- Cette logique permet de centraliser le retrait des tuiles et leurs conséquences dans une seule classe. L'interface graphique se contente ensuite d'afficher le résultat de l'action au joueur.

3. Gestion des créatures (CreatureManager)

Le CreatureManager orchestre le lancer du dé et le déplacement des créatures selon leurs règles propres. La distance maximale de déplacement est codée dans chaque instance de Creature: 1 case pour le serpent, 2 pour le requin, 3 pour la baleine.

```
public ActionResult moveCreature (GameState gs,
    Creature creature, HexCoordinate target) {
    // Validation distance par BFS
    int dist = bfsDistance (board, creature.getPosition(), target);
    if (dist < 1 || dist > creature.getMaxMoves())
        return ActionResult.failure ("Distance invalide.");
    creature.setPosition (target);
    // Application des effets selon le type
    return switch (creature.getType()) {
        case SERPENT -> applySerpent (gs, target);
        case REQUIN -> applyShark (gs, target);
        case BALEINE -> applyWhale (gs, creature, target);
    };
}
```

Figure 3: Déplacement et attaque d'une créature (CreatureManager)

Les effets respectent fidèlement les règles du sujet :

- Serpent : élimine nageurs et bateaux occupés ; les bateaux vides restent.
- Requin : élimine uniquement les nageurs ; ignore les bateaux.
- Baleine : chavire les bateaux occupés ; si un requin est présent sur la même case, les passagers devenus nageurs sont aussi éliminés.

4. Tuiles en main (InteractionController)

La logique de jeu des tuiles rouges est intégrée directement dans l'InteractionController, exploitant les états de sélection étendus de l'énumération SelectionKind:

- HAND_DOLPHIN_SWIMMER/ HAND_DOLPHIN_TARGET: sélection du nageur puis de la destination.
- HAND_WIND_BOAT/ HAND_WIND_TARGET: sélection du bateau puis de la destination.
- HAND_CREATURE/ HAND_CREATURE_TARGET: sélection de la créature puis de la case de mer cible.
- CANCEL_SHARK/ CANCEL_WHALE: tuiles de défense ; un message informe le joueur qu'elles se déclenchent automatiquement lors d'une attaque.

V. Interface graphique (Swing)

1. Écran d'accueil (WelcomeFrame)

Au lancement, WelcomeFrame affiche un menu principal avec fond dégradé maritime. Trois boutons sont proposés : Nouvelle partie (lance directement une partie à 4 joueurs), Règles du jeu (affiche les règles dans un panneau intégré) et Quitter. Aucun choix du nombre de joueurs n'est nécessaire : la partie se lance toujours à 4 joueurs, conformément à l'énoncé.

2. Plateau de jeu (InteractiveBoardPanel)

L'affichage du plateau est assuré par InteractiveBoardPanel, qui rend les cellules hexagonales par construction géométrique de polygones en Java2D, avec anti-aliasing activé. Les éléments affichés :

- Cases colorées selon leur type (mer bleue, plage jaune, forêt verte, montagne grise, abris beige).
- Explorateurs et nageurs représentés par des disques colorés (rouge, bleu, vert, jaune).
- Bateaux représentés par un rectangle brun arrondi.
- Créatures marines représentées par des ovales : S (serpent, violet), R (requin, bleu), B (baleine, cyan).
- Surlignage rouge des tuiles retirables en phase 3.
- Surlignage orange du pion sélectionné.
- Marqueur spiralé violet sur les cases de départ des serpents.

3. Contrôleur d'interaction (InteractionController)

L'InteractionController coordonne l'ensemble des interactions utilisateur au fil des phases décrites par DemoPhase:

- ❖ SETUP_EXPLORERS: placement des explorateurs.
- ❖ SETUP_BOATS: placement des bateaux.
- ❖ PLAY_HAND_TILE: jouer une tuile de la main (optionnel).
- ❖ MOVEMENT: 3 déplacements.
- ❖ TILE_REMOVAL: retrait obligatoire d'une tuile.
- ❖ CREATURE_DICE: lancer le dé.
- ❖ CREATURE_MOVE: déplacer la créature.
- ❖ GAME_OVER: fin de partie, affichage des scores.

4. Panneau d'information et aide

Un panneau latéral affiche en temps réel l'état du tour : joueur actif, phase courante, points de

déplacement restants, statistiques par joueur (explorateurs sauvés, nageurs, sur l'île, dans les bateaux, tuiles en main, score), et un rappel des règles principales. Un bouton dédié permet d'accéder aux règles complètes depuis l'écran d'accueil.

5. Bouton Placement automatique

Un bouton Placement automatique est disponible pendant les phases de préparation. Il place tous les explorateurs et bateaux de façon aléatoire valide en un seul clic, permettant de démarrer rapidement une partie.

VI. Illustrations de l'interface

Cette section présente des captures d'écran de l'interface du jeu, illustrant les différentes phases de la partie.

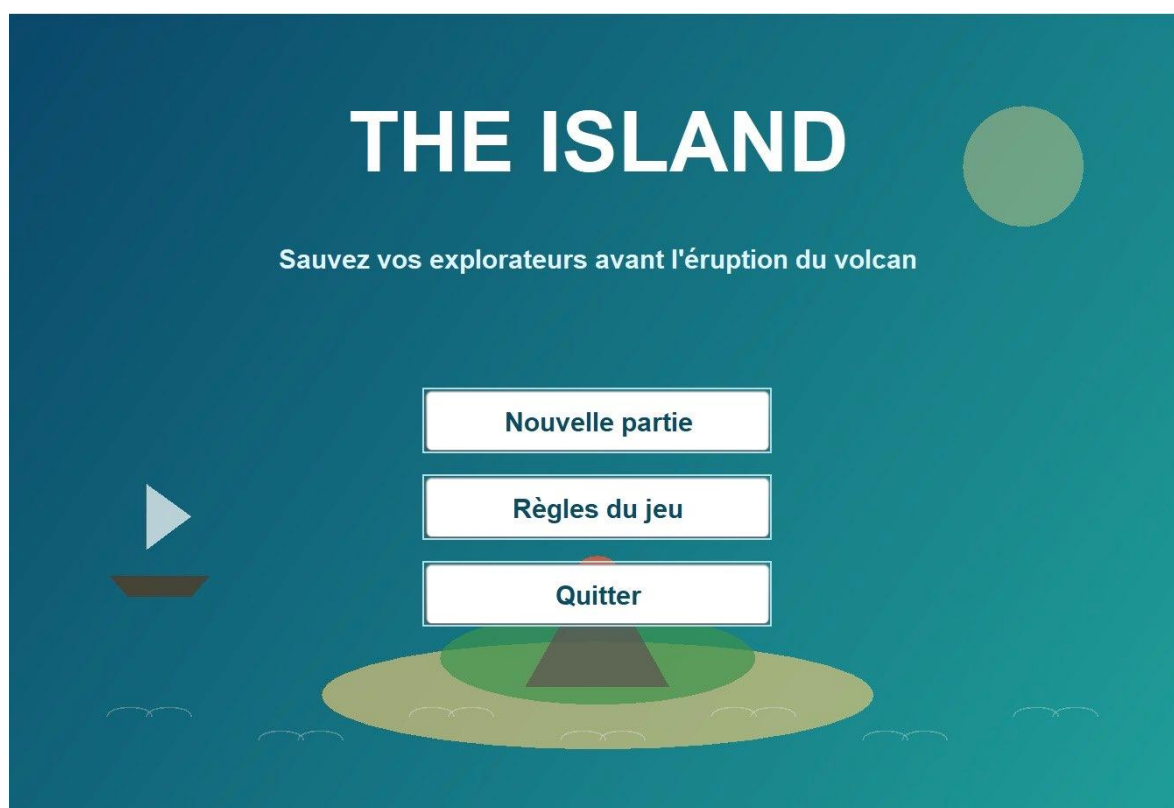


Figure 4:Écran d'accueil (WelcomeFrame)

The Island

Phase : Placement des explorateurs

Joueur actif : Joueur 1 (Rouge)

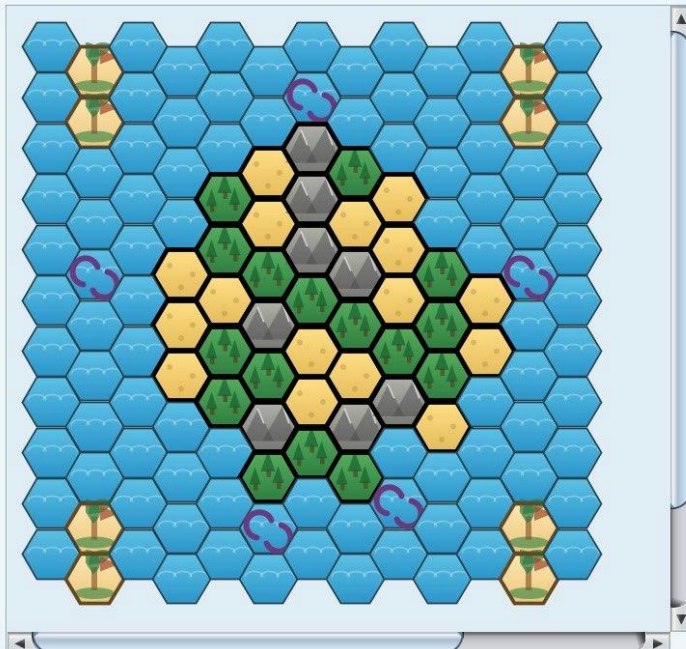
Prochain explorateur à placer : P1-E1 | valeur à mémoriser : 1.

Message : Préparation : Joueur 1 place son premier explorateur.

Aide rapide

Placement automatique

Annuler sélection



Infos de partie

Résumé

Explorateurs

0

Bateaux

0

Nageurs

0

Sauvés

0

Partie

Tuiles restantes : 40

Plages : 16 Forêts : 16 Montagnes : 8

Créatures en jeu : 0

Stock : 6 5

Joueurs

Figure 5: Plateau de jeu pendant une phase de déplacement

The Island

Phase : 1/4 · Jouer une tuile de la main

Joueur actif : Joueur 1 (Rouge)

Aucune tuile en main.

Message : Tour de Joueur 1 — Phase 1 : jouer une tuile (ou Passer).

Aide rapide

Passer (pas de tuile)

Annuler sélection



Infos de partie

Résumé

Explorateurs

40

Bateaux

8

Nageurs

0

Sauvés

0

Partie

Tuiles restantes : 40

Plages : 16 Forêts : 16 Montagnes : 8

Créatures en jeu : 5

Stock : 6 5

Joueurs

Figure 6: Lancer du dé de créature

The Island

Phase : Fin de partie — Scores finaux

Joueur actif : Joueur 2 (Bleu)

Message : Tuile Montagne retirée. 🌋 VOLCAN ! Fin de la partie !

Aide rapide



Infos de partie

Joueur 4

✓ Sauvés : 0 🐬 Nageurs : 0
🗺️ Tile : 0 🚢 Bateau : 0 🧰 Main : 2 ★ Score : 0

Scores finaux

🏆 Joueur 1 : 1 pts (1 sauvés)

• Joueur 2 : 0 pts

• Joueur 3 : 0 pts

• Joueur 4 : 0 pts

Gagnant : Joueur 1

Rappels

- 3 déplacements par tour
- retrait : plage → forêt → montagne
- requin/serpent : tue les nageurs
- baleine : chavire les bateaux occupés
- volcan : fin immédiate

Figure 7: Fin de partie et affichage des scores

Les captures d'écran illustrent les principales interactions : placement des explorateurs, déplacement des pions, retrait des tuiles, déplacement des créatures et calcul des scores en fin de partie.

VII. Tests unitaires

Le projet contient des tests unitaires réalisés avec JUnit 5. Ces tests permettent de vérifier plusieurs éléments importants du jeu, notamment l'initialisation du plateau, la création des joueurs, le placement initial et certaines règles de déplacement.

Le projet contient 4 classes de tests, avec 16 tests unitaires au total.

Tableau 3: Suites de tests JUnit 5

Classe de test	Nombre de tests	Élément testé
BoardInitializationTest	5	Plateau, coordonnées et types de cases
GameSetupFactoryTest	2	Création des joueurs, explorateurs et bateaux
InitialPlacementServiceTest	3	Placement initial des explorateurs et des bateaux
MovementServiceTest	6	Déplacements, bateaux et règles associées

Ces tests vérifient à la fois des cas valides et des cas invalides. Ils permettent de contrôler que les principales règles du modèle sont respectées, par exemple la création correcte du plateau, le nombre d'explorateurs par joueur, la capacité des bateaux et certaines validations de déplacement.

VIII. Manuel utilisateur

1. Prérequis et lancement

Le projet requiert Java 17 ou supérieur et Maven 3.6+.

```
mvn clean package -DskipTests
java -jar target/the-island-0.3.0-SNAPSHOT.jar
```

2. Démarrage du jeu

Au lancement, l'écran d'accueil apparaît. Cliquer Nouvelle partie lance immédiatement une partie à 4 joueurs humains sur la même machine, sans configuration supplémentaire.

3. Déroulement d'une partie

a. Préparation – Placement des explorateurs

Chaque joueur place, à tour de rôle, ses 10 explorateurs sur les tuiles de terrain disponibles en cliquant sur la case cible. La valeur du trésor de l'explorateur à placer est affichée : il est important de la mémoriser car les pions seront retournés face cachée. Le bouton Placement automatique permet de tout placer d'un seul clic.

b. Préparation – Placement des bateaux

Chaque joueur place ses 2 bateaux sur une case de mer adjacente à une tuile de terrain. Le placement automatique gère aussi cette phase.

c. Tour de jeu (Phase 1 – Tuile de la main)

Si le joueur possède des tuiles rouges, un bouton Jouer une tuile apparaît. Il choisit la tuile, puis :

- ❖ Dauphin : cliquer un nageur, puis sa destination (1–3 cases de mer).
- ❖ Vent : cliquer un bateau contrôlé, puis sa destination (1–3 cases).
- ❖ Déplacer créature : cliquer la créature cible, puis n'importe quelle case de mer.
- ❖ Défense requin/baleine : ces tuiles se jouent automatiquement hors tour.

Le bouton Passer permet de sauter cette phase.

d. Tour de jeu (Phase 2 – Déplacements)

Le joueur dispose de 3 points de déplacement. Un clic sélectionne le pion (surlignage orange), un second clic indique la destination. Bouton Sauter dans l'eau disponible si un passager est sélectionné. Cliquer Retirer une tuile pour terminer les déplacements.

e. Tour de jeu (Phase 3 – Retrait de tuile)

Les tuiles retirables sont surlignées en rouge (ordre imposé : plage → forêt → montagne). Cliquer une tuile pour la retirer ; l'effet caché s'applique automatiquement.

f. Tour de jeu (Phase 4 – Dé de créature)

Cliquer Lancer le dé. Le type de créature est affiché. Cliquer la créature correspondante sur le plateau, puis sa case de destination. Cliquer Passer pour ne pas déplacer de créature.

g. Fin de partie

Dès que le volcan est révélé, la partie s'arrête. Les scores s'affichent : somme des valeurs de trésor des explorateurs sauvés. En cas d'égalité, le joueur ayant sauvé le plus d'explorateurs gagne.

IX. Planning et répartition des tâches**1. Découpage en phases de développement**

Tableau 4: Phases de développement

Phase	Périmètre	Durée estimée
Phase 1	Modèle de domaine	15 jours
Phase 2	Services de setup	10 jours
Phase 3	Déplacements	15 jours
Phase 4	Moteur de jeu (créatures, tuiles)	15 jours
Phase 5	Interface graphique Swing	25 jours
Phase 6	Tests unitaires	05 jours
Phase 7	Packaging, Javadoc, rapport	05 jours

2. Répartition des tâches par membre

Membre	Responsabilités principales
Najlae BETTACHE (Chef de projet)	Coordination générale, architecture logicielle, intégration finale, packaging Maven, rédaction du rapport.
Nathanaël FOUODJI DJOUDA	Service de déplacement MovementContext, MovementService, BoatControlService, ActionResult.
Boubacar BOULKASSOUM	Modèle objet : Explorer, Boat, Player, GameState, PieceState, énumérations. Rédaction des tests unitaires JUnit 5.
Mamadou Sadio DIALLO	Interface graphique : InteractiveBoardPanel (rendu hexagonal Java2D, pions, créatures), WelcomeFrame, panneau d'aide.
Uriel Ange Javituriel	Moteur de jeu : TileRemovalService, CreatureManager, Dice, effets de tuiles (TileEffect), InteractionController

3. Outils de développement

- ❖ Environnement de développement : IntelliJ IDEA 2024
- ❖ Gestion de version : Git
- ❖ Build : Apache Maven 3.9
- ❖ Framework de tests : JUnit Jupiter 5.10.2
- ❖ Java cible : OpenJDK 17

X. Problèmes rencontrés et solutions apportées

- ❖ Voisins hexagonaux : la topologie hexagonale avec offset pair/impair nécessite deux ensembles de deltas selon la parité de la colonne. La classe Board implémente ces deux cas, garantissant une topologie correcte pour tous les déplacements et calculs de distance (BFS).
- ❖ Contrôle des bateaux par majorité : en cas d'égalité (même nombre d'explorateurs de deux joueurs différents), chaque joueur concerné peut contrôler le bateau, conformément au sujet. Cette logique est isolée dans BoatControlService.
- ❖ Effet « Bateau apparaît » avec plus de 3 nageurs : quand la tuile fait apparaître un bateau sur une case contenant plus de 3 nageurs, le sujet précise que le joueur ayant révélé la tuile choisit lesquels montent. Un dialogue interactif (JOptionPane) gère ce cas.
- ❖ Synchronisation des phases : l'enchaînement des 8 phases du tour est géré par le pattern état implicite de l'énumération DemoPhase dans l'InteractionController. L'énumération SelectionKind affine les sous-états internes de chaque phase (notamment pour les tuiles en main).
- ❖ Rendu hexagonal : la conversion entre coordonnées pixel et coordonnées hexagonales utilise un offset standard. La grille hexagonale est reconstruite à chaque paintComponent et les polygones sont stockés dans une Map pour la détection des clics.

XI. Bilan

1. Fonctionnalités implémentées

- ❖ Plateau hexagonal avec îles-refuges aux quatre coins et emplacements de départ des serpents.
- ❖ 40 tuiles de terrain mélangées avec effets cachés, distribuées selon la répartition officielle.
- ❖ 4 joueurs humains sur la même machine ; tours alternants dans l'ordre des aiguilles d'une montre.
- ❖ Placement manuel ou automatique (bouton Placement automatique disponible dans l'interface).
- ❖ Déplacements complets (3 points par tour) : terrain, nageurs, bateaux, passagers, sauvetage.
- ❖ Retrait des tuiles dans l'ordre obligatoire (plage → forêt → montagne).
- ❖ Tuiles immédiates : requin, baleine, bateau, tourbillon, volcan.
- ❖ Tuiles en main jouables : dauphin, vent, déplacer serpent/requin/baleine.
- ❖ Tuiles de défense : CANCEL_SHARK et CANCEL_WHALE.
- ❖ Dé de créature et déplacement des serpents, requins et baleines avec effets complets.
- ❖ Fin de partie (volcan) et calcul des scores avec départage.
- ❖ Interface graphique Swing avec écran d'accueil, plateau interactif, panneau de statistiques et règles du jeu.
- ❖ 16 tests unitaires JUnit 5 couvrant le modèle, l'initialisation et les services de déplacement.

2. Qualité du code

- ❖ Conventions de codage SUN respectées (nommage camelCase/PascalCase, constantes en UPPER_CASE).
- ❖ Toutes les classes et méthodes publiques documentées par Javadoc.
- ❖ Utilisation des collections Java (List, Map, Optional, Stream).
- ❖ Collections exposées uniquement via Collections.unmodifiableList().
- ❖ Gestion stricte des exceptions (IllegalStateException, IllegalArgumentException, NullPointerException).
- ❖ 5 126 lignes de code au total (hors tests).

Conclusion

Le projet *The Island* a été mené à bien dans son intégralité. Toutes les règles décrites dans l'énoncé ont été implémentées, des missions de base (plateau, déplacements, sauvetage) jusqu'aux mécaniques avancées (créatures marines, effets de tuiles, tuiles en main, défense hors tour, volcan et scores finaux). La partie se joue à 4 joueurs humains sur une seule machine, conformément à l'énoncé. L'interface Swing guide le joueur à chaque phase du tour grâce à des messages contextuels, un surlignage des actions disponibles et des rappels de règles intégrés. Sur le plan technique, toutes les contraintes imposées sont respectées : Java 17, Swing, Maven, JAR exécutable, conventions SUN et Javadoc. La séparation en package model, movement, setup, engine et ui assure une bonne maintenabilité. Ce projet nous a permis de mettre en pratique les principes fondamentaux de l'orienter objet (encapsulation, séparation des responsabilités, composition) dans un contexte concret, complet et ludique.